

Learning resources for DevOps Engineers

1. GIT

All your resources (files) will be held in a GIT repository. Those files are application code but also infrastructure as a code.

Git is a free tool used for source code management. Git is used to tracking changes in the source code, enabling multiple developers to work together on non-linear development.

The two most popular Git platforms are GitLab and GitHub.

Here you need to learn Git commands, like git clone, branch, merge, and how to collaborate on a project with pull requests.

Resources:

- [Pro Git Book](#) FREE
- [Learn Git by Atlassian](#) FREE
- [Learn Git Branching](#) FREE
- [Learn Git & GitHub on CodeAcademy](#) FREE
- [Git Command Explorer](#) FREE
- [Git Immersion](#) FREE
- [A Visual Git Reference](#) FREE
- [Git & GitHub Tutorial](#) FREE
- [Advanced Git Tutorial](#) FREE

2. Learn one programming language

As an engineer, it is recommended to know at least one programming language that you can use to write automation scripts.

Some popular programming languages for DevOps-es are Python, Go, and JavaScript.

Python is a multi-paradigm language. Being an interpreted language, code is executed as soon as it is written, and the syntax allows for writing code in different ways. Python is frequently recommended as the first language new coders should learn, because of its focus on readability, consistency, and ease of use.

Here you need to learn basic concepts of programming languages, such as syntax, if/else, loops, data structures, etc.

Resources:

- Python:
 - [Automate the Boring Stuff with Python book](#) FREE
 - [Python Crash Course](#) FREE
- JavaScript:
 - [The Modern JavaScript Tutorial](#) FREE
 - [JavaScript Crash Course For Beginners](#) FREE
 - [Eloquent JavaScript, 3rd edition](#), Marjin Haverbeke FREE book
- Go
 - [Go by Example](#) FREE
 - [Learn Go with Tests](#) FREE

3. Learn Linux & Scripting

An Operating system serves as a bridge between a computer's user and its hardware. Its function is to offer a setting in which a user can conveniently and effectively run programs.

As most servers use Linux OS, you need to make yourself comfortable with Linux and its CLI.

Learn basic commands, such as: ls, cd, mkdir, rm, cp, mv, touch, cat, printenv, grep, find, chmod, chown, ps, kill, top, df, du, tar, gzip, ssh, scp, wget, curl, etc.

One easy-to-start distribution is Ubuntu.

In addition, you need to know scripting to automate tasks for development and operations.

Here you can learn OS-specific languages, such as Bash or Powershell, or independent, like Python or Go.

Resources:

- [Operating System - Overview](#) FREE
- [Shell Scripting Tutorial](#) FREE
- [Powershell Tutorial for Beginners: Learn Powershell Scripting](#) FREE
- [Bash Reference Manual](#) FREE
- [Ultimate Guide: Getting Started With Ubuntu](#) FREE
- [FreeBSD Handbook](#) FREE
- [Linux command handbook](#) FREE
- [Linux commands for Cloud and Devops engineers](#) FREE

4. Learn Networking & Security

A network protocol is an established set of rules that determine how data is transmitted between different devices in the same network. Essentially, it allows connected devices to communicate with each other, regardless of any differences in their internal processes, structure, or design.

Here you will need to know how a network works, how to configure firewalls, understand how DNS works, OSI model, IP addresses, ports, etc.

Resources:

- [OSI Model Explained](#) FREE
- [Computer Networking: A Top-Down Approach](#) book [Video Content](#) video
- [TCP/IP and Networking Fundamentals for IT Pros](#) Pluralsight course
- [DevSecOps : Master Securing CI/CD | DevOps Pipeline](#) Udemy course
- [Hands-On Security in DevOps: Ensure continuous security, deployment, and delivery with DevSecOps](#) Book
- [Securing DevOps: Security in the Cloud](#) Book
- [Professor Messers Network+ course](#) FREE
- [How DNS works](#)
- [How DNSSEC works](#)
- [How HTTPS works](#)

5. Learn Server Management

Server management includes all the infrastructure monitoring and maintenance required for servers to operate reliably and at optimal performance levels. The primary goals of an effective server management strategy are to:

- Minimize server slowdowns and downtime while maximizing reliability.

- Build secure server environments.
- Scale servers and related operations to meet the needs of the organization over time.

Here you will need to know what is forward and reverse proxies, caching servers, and how to operate Web Servers, such as Nginx, Apache, or IIS.

Resources:

- [What is a reverse proxy?](#) FREE
- [What is a CDN edge server??](#) FREE
- [Cache Server](#) FREE
- [Reverse Proxy vs. Forward Proxy: The Differences](#) FREE
- [What is load balancing?](#) FREE
- [What is a Firewall?](#) FREE
- [The NGINX Handbook](#) FREE
- [Learn Apache Server](#) FREE
- [Learn IIS](#) FREE

6. Learn Containers

A container is a standard unit of software that packages up code and all its dependencies, so the application runs quickly and reliably from one computing environment to another.

Docker is by far the most popular container technology today. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries, and settings.

Here you need to know how to run containers, Docker Networking, Volumes, Dockerfiles, and run multiple containers with Docker-Compose.

Docker Compose is important as pre-requisite for Kubernetes. It is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration.

Resources:

- [What are Containers?](#) FREE
- [Learning Containers From The Bottom Up](#) FREE

- [Docker Crash Course for Absolute Beginners by TechWorld with Nana](#) FREE
- [Docker Tutorial for Beginners by TechWorld with Nana](#) FREE
- [Ultimate Docker Compose Tutorial by TechWorld with Nana](#) FREE
- [Docker Mastery: with Kubernetes + Swarm from a Docker Captain](#) Udemy course
- [What is Service Mesh?](#) FREE
- [DevOps with Kubernetes](#) FREE
- [OCI Specification](#)

7. Learn Container Orchestration

Container orchestration automates the deployment, management, scaling, and networking of containers.

Container orchestration can be used in any environment where you use containers. It can help you to deploy the same application across different environments without needing to redesign it. And microservices in containers make it easier to orchestrate services, including storage, networking, and security.

Here you need to learn how Kubernetes works, and how to administer the Kubernetes cluster and deploy applications on it.

You need to know basic components of Kubernetes, such as: Master Node, Worker Node, Pod, ReplicaSet, Deployment, Service, Ingress, ConfigMap, Secret, PersistentVolume, PersistentVolumeClaim, StatefulSet, DaemonSet, Job and CronJob.

Also, you need to know how to work with kubectl and Helm tools.

Resources:

- [Kubernetes Crash Course for Absolute Beginners by TechWorld with Nana](#) FREE
- [Primer: How Kubernetes Came to Be, What It Is, and Why You Should Care](#) Article
- [Certified Kubernetes Administrator \(CKA\) with Practice Tests](#) Udemy course
- [Learn Kubernetes - Beginners to Advanced by KodeKloud](#) Course
- [Understand when to use Cluster Services, Ingresses or API Gateways](#) FREE
- [Understand which Problems Service Mesh solve \(Use an Abstraction smi-spec.io\)](#) FREE
- [Learn how to automate TLS and DNS](#) FREE
- [Kubernetes Up and Running](#) Book
- [Kubernetes Learning Path - 50 days from zero to hero from Microsoft](#) FREE
- [Below Kubernetes: Demystifying container runtimes](#) FREE

8. Learn Infrastructure as a code (X as Code)

Sometimes referred to as IaC, it refers to the techniques and tools used to define infrastructure, typically in a markup language like YAML or JSON. Infrastructure as code allows Engineers to automate environment setup and teardown. Accelerates and de-risks deployment by provisioning gold copy environments on demand.

Terraform is the most popular infrastructure provisioning tool, but there are others such as Ansible, Chef, Puppet, and Vagrant.

Here you need to know how to do infrastructure provisioning and configuration management.

Resources:

- [GUIs, CLI, APIs: Learn Basic Terms of Infrastructure-as-Code](#) FREE
- Terraform:
 - [Official Terraform Tutorials](#) FREE
 - [A Comprehensive Guide to Terraform](#) FREE
 - [Automate Terraform documentation like a pro!](#) FREE
 - [Writing reusable Terraform modules](#) FREE
 - [Terraform Course - Automate your AWS cloud infrastructure](#) FREE
 - [HashiCorp Terraform Associate Certification Course](#) FREE
 - [Terraform on Azure](#) FREE
- Puppet:
 - [Puppet overview](#) FREE
 - [Puppet Courses](#) FREE and PAID
- Chef:
 - [Learn Chef](#) FREE
- Ansible:
 - [Getting Started With Ansible](#) FREE
 - [Learning Ansible Basics](#)
 - [Get started with Red Hat Ansible](#) FREE and PAID
 - [Mastering Ansible](#) Udemy Course
- Istio:
 - [What is Istio?](#) FREE

9. Learn CI/CD

Continuous Integration / Continuous Deployment (CI/CD) is a method to frequently deliver apps to customers by introducing automation into the stages of app

development. CI/CD is a solution to the problems integrating new code can cause for development and operations teams.

CI/CD introduces continuous automation and continuous monitoring throughout the lifecycle of apps, from integration and testing phases to delivery and deployment. These connected practices are often referred to as a "CI/CD pipeline" and are supported by development and operations teams.

There are different stages of a CI/CD pipeline, such as: build, test and deploy, but there could be much more activities included:

- Checking code from version control and building it
- Having staged gates for different kinds of approvals
- Managing environment variables
- Restarting services
- Executing tests
- And more...

Here you need to learn how to set up CI/CD server, integrate code and trigger pipelines automatically, store and read secrets, and build and package management tools.

Some popular CI/CD tools are: Jenkins, TeamCity, CircleCI, Bamboo, GitLab, and Azure DevOps.

Resources:

- [Continuous Integration](#) FREE
- [CI/CD Pipeline: A Gentle Introduction](#) FREE
- Jenkins:
 - [Jenkins, From Zero To Hero: Become a DevOps Jenkins Master](#) Udemy course
- Azure DevOps:
 - [Learn Azure DevOps](#) FREE
- GitHub Actions:
 - [Learn GitHub actions](#) FREE
 - [GitHub Actions Tutorial by Tech World with Nana](#) FREE
 - [Workflow syntax for GitHub Actions](#) FREE
- GitLab:
 - [Learn GitLab with tutorials](#) FREE
 - [GitLab CI CD Tutorial for Beginners by Tech World With Nana](#) FREE
 - [Get started with GitLab CI/CD](#) FREE
 - [GitLab Cheatsheets](#) FREE

10. Learn Monitoring & Observability

Monitoring entails overseeing the entire development process from planning, development, integration and testing, deployment, and operations. It involves a complete and real-time view of the status of applications, services, and infrastructure in the production environment.

This is especially important when our software is in production, and we need to track all kinds of issues in our infrastructure and application.

The two most popular tools are Prometheus and Grafana, but also Cloud-based tools such as AWS CloudWatch, Azure Monitor, and Google Cloud Monitoring.

Here you need to know how to set up monitoring and visualize data, creating and setting up alerting, and creating automatization during alerting.

Resources:

- [What Is Observability? Comprehensive Beginners Guide](#) FREE
- [The Hows, Whys and Whats of Monitoring Microservices](#) FREE
- [DevOps Monitoring](#) FREE
- [Applying Basic vs. Advanced Monitoring Techniques](#) FREE
- [Learn Prometheus](#) FREE
- [Learn Grafana](#) FREE
- [Beautiful Dashboards with Grafana and Prometheus](#) FREE
- [Elastic Stack](#) FREE
- [AWS Tutorial - Amazon CloudWatch Tutorial](#) FREE
- [Datadog 101 Course](#) FREE
- [Splunk Fundamentals](#) FREE

11. Learn one Cloud provider

Cloud providers provide a layer of APIs to abstract infrastructure and provision it based on security and billing boundaries. The cloud runs on servers in data centers, but the abstractions cleverly give the appearance of interacting with a single "platform" or large application. The ability to quickly provision, configure and secure resources with cloud providers has been key to both the tremendous success, and complexity, of modern DevOps.

The most popular cloud providers in the market are AWS and Azure, as well as Google Cloud.

Here you need to know how to manage users and administration, networks, virtual servers, etc.

Resources:

- [Serverless 101 - Serverless Land](#) FREE
- Azure:
 - [Exam AZ-900: Microsoft Azure Fundamentals](#) FREE
 - [Microsoft Azure Fundamentals Certification Course \(AZ-900\)](#) FREE
 - [AZ-900 | Microsoft Azure Fundamentals Full Course, Free Practice Tests, Website and Study Guides](#) FREE
- AWS:
 - [Ultimate AWS Certified Cloud Practitioner - 2022](#) Udemy
 - [AWS Developer by A Cloud Guru](#) Learning path
 - [AWS Well-Architected](#) FREE
- Google Cloud:
 - [Google Cloud Associate Cloud Engineer Course](#) FREE
 - [Google Cloud Well-Architected Framework](#) FREE

12. Learn Software Engineering Practices

As a DevOps engineer, you will probably work in a team with other developers in an Agile world, such as Scrum. So, it is very important to know different parts of SDLC, as well as the tools which are used there.

In addition, it would be good to know how automation testing is working, as you will need to set up it in CI/CD way.

Here you need to know what is Scrum, all phases of SDLC, how automation testing works, etc.

Resources:

- [What is Scrum?](#) FREE
- [Ways To Learn About Scrum](#) FREE
- [Software Development Life Cycle \(SDLC\) Phases & Models](#) FREE
- [The Beginner's Guide to Agile in Jira: Course description](#) FREE
- [Learn SAFe](#) FREE
- [Learn Automation Testing](#) FREE
- [GitLab - Beginner's Guide to DevOps](#) FREE
- [Common SDLC Models](#) FREE

Bonus: Learn DevSecOps Fundamentals

Security must be integrated throughout the DevOps lifecycle rather than added as an afterthought.

Here you will need to learn how to integrate security into the DevOps pipeline, and how to automate security testing (SAST and DAST).

Also, you need to know how to manage secrets and credentials, and how to set up security policies.

Resources:

- [OWASP DevSecOps Guideline](#) FREE
- [Supply Chain Levels for Software Artifacts \(SLSA\)](#) FREE
- [HashiCorp Vault Documentation](#) FREE
- [Trivy Documentation](#) FREE
- [Falco Runtime Security](#) FREE
- [DevSecOps: A leader's guide](#) FREE
- [Container Security](#) book

Additional resources

Tools

- Work Tracking:** [Asana](#), [Monday](#), [Jira](#), [Trello](#), [Azure Boards](#).
- Source code control: [Git](#), [Github](#), [GitLab](#), [BitBucket](#), [Azure DevOps](#).
- CI/CD: [Jenkins](#), [Team City](#), [Github Actions](#), [Travis CI](#), [Bamboo](#), [Circle CI](#), [Azure Pipelines](#), [Octopus Deploy](#), [Harness](#), [CloudBees CodeShip](#).
- Source Code Analysis: [SonarQube](#), [Veracode](#).
- Artifact management: [Artifactory](#), [Docker Container Register](#), [npm](#), [Yarn](#), [NuGet](#).
- Configuration Management: [Terraform](#), [Ansible](#), [Puppet](#), [Chef](#).
- Container orchestration: [Docker](#), [Kubernetes](#), [Red Hat OpenShift](#).
- Monitoring: [Prometheus](#), [Grafana](#), [Splunk](#), [Dynatrace](#), [Kibana](#).